R Course
Assignment 2

Introduction

In this assignment, you will debug some R functions for working with DNA sequences.

On ILIAS, you will find `assignment2_functions.R` and `assignment2_demo.R`. The functions file contains faulty code. The demo file runs small tests and shows the expected results.

The Functions

1. `gc_content(seq)` should return the percentage of bases in a DNA sequence that are G or C (e.g. `gc_content("GCGCATAT")` should return 50).

```r
# Returns the GC content as a percentage.
gc_content <- function(seq) {
  bases <- strsplit(seq, "")[[1]]
  gc_count <- sum(bases == "G" & bases == "C")
  return(gc_count / length(bases))
}
```

2. `reverse_complement(seq)` should replace A with T, T with A, C with G and G with C, and then reverse the sequence (e.g. `reverse_complement(ÄTGC")` should return `"GCAT"`).

```r
# Returns the reverse complement of a DNA sequence.
reverse_complement <- function(seq) {
  bases <- strsplit(seq, "")[[1]]

  bases[bases == "A"] <- "T"
  bases[bases == "T"] <- "A"
  bases[bases == "C"] <- "G"
  bases[bases == "G"] <- "C"

  reversed_bases <- rev(bases)
  paste(reversed_bases, collapse = "")
}
```

3. `transcribe(dna_seq)` should replace every T by U (e.g. `transcribe(ÄTGCGT")` should return `ÄUGCGU"`).

```r
# Transcribes DNA into RNA.
transcribe <- function(dna_seq) {
  bases <- strsplit(dna_seq, "")[[1]]
  bases[bases == "U"] <- "T"
  paste(bases, collapse = "")
}
```

4. `translate(rna_seq)` should translate an RNA sequence into amino acids, reading three bases (one codon) at a time. Stop and return the result immediately when you reach the stop codon UAA. Only the following six codons need to be supported:

- AUG → M
- GCU → A
- GGG → G
- UUU → F
- CCC → P
- UAA → Stop

Example: `translate(ÄUGGCUUAAGGG")` should return `MMA"`.

```
# Translates an RNA sequence until a stop codon.
translate <- function(rna_seq) {
  codon_translations <- c(AUG = "M", GCU = "A", UAA = "*", GGG = "G",
                    UUU = "F", CCC = "P")
  protein <- ""
  for (i in 1:nchar(rna_seq)) {
    codon <- substr(rna_seq, i, i + 2)
    amino_acid <- codon_translations[codon]
    protein <- paste0(protein, amino_acid)
  }
  return(protein)
}
```

5. `hardy_weinberg(p)` should return the genotype frequencies $AA = p^2$, $Aa = 2p(1-p)$ and $aa = (1-p)^2$. `hardy_weinberg(0.6)` should return 0.36, 0.48 and 0.16.

```
# Returns Hardy-Weinberg genotype frequencies.
hardy_weinberg <- function(p) {
  q <- 1 - p
  AA <- p^2
  Aa <- p * q
  aa <- q^2
  return(AA)
}
```

6. `random_dna(length)` should return a random DNA sequence of the requested length, using only A, T, C and G.

```
# Returns a random DNA sequence as a character vector.
random_dna <- function(length) {
  bases <- c("A", "T", "C", "G")
  sample(bases, length, replace = FALSE)
}
```

Your Task

1. Download both files from ILIAS and run `assignment2_demo.R`. Compare the actual outputs with the expected outputs.

1a)

```r
gc_content("GCGCATAT")
```

```
## [1] 0
```

```r
# expected result: 50
```

1b)

```r
reverse_complement("ATGC")
```

```
## [1] "CCAA"
```

```r
# expected result: "GCAT"
```

1c)

```r
transcribe("ATGCGT")
```

```
## [1] "ATGCGT"
```

```r
# expected result: "AUGCGU"
```

1d)

```r
translate("AUGGCUUAAGGG")
```

```
## [1] "MNANAANANA*NANAGNANA"
```

```r
# expected result: "MA"
```

1e)

```r
hardy_weinberg(0.6)
```

```
## [1] 0.36
```

```r
# expected result: a list (or vector) with components
# AA (0.36), Aa (0.48), and aa (0.16)
```

1f)

```r
set.seed(10)
random_sequence <- random_dna(20)
```

```
## Error in `sample.int()`:
## ! cannot take a sample larger than the population when 'replace = FALSE'
```

```
random_sequence
```

```
## Error:
## ! object 'random_sequence' not found
```

```
# expected result: 20 characters sampled from {"A", "T", "C", "G"}


# Add one additional test for each function after fixing the code.
```

2. Fix the bugs in `assignment2_functions.R`, one function at a time. Add comments above each corrected section explaining the bug. Keep the overall structure and test after every change.

3. Add one additional test case of your own for each function.